

Comparative Analysis of Image Classification Techniques and Optimizers on the Fashion-MNIST Dataset

Adham HAMDY and Mohammad ABELAZIM
Sorbonne University, Department of Automation and Robotics

10/2025



dataset consists of 60,000 training images and 10,000 test images, with each image being a 28x28 grayscale representation of an article of clothing from one of 10 classes.

A wide variety of techniques has been applied to this dataset. These range from traditional machine learning methods, such as k-Nearest Neighbors (k-NN) and Histogram of Oriented Gradients (HOG) feature descriptors paired with a Support Vector Machine (SVM), to advanced deep learning models, including Artificial Neural Networks (ANN), Deep Residual Networks (ResNet), and Convolutional Neural Networks (CNN).

Furthermore, the performance of these deep learning models is heavily influenced by the choice of optimization algorithm. Common optimizers like Stochastic Gradient Descent (SGD), Adagrad, RMSprop, and Adam can produce significantly different results even with the same model architecture.

Abstract

Image classification is a fundamental challenge in computer vision. A popular and modern benchmark for this task is the Fashion-MNIST dataset, which provides a more complex challenge than the traditional MNIST. This dataset consists of 60,000 training images and 10,000 test images, with each image being a 28x28 grayscale representation of an article of clothing from one of 10 classes.

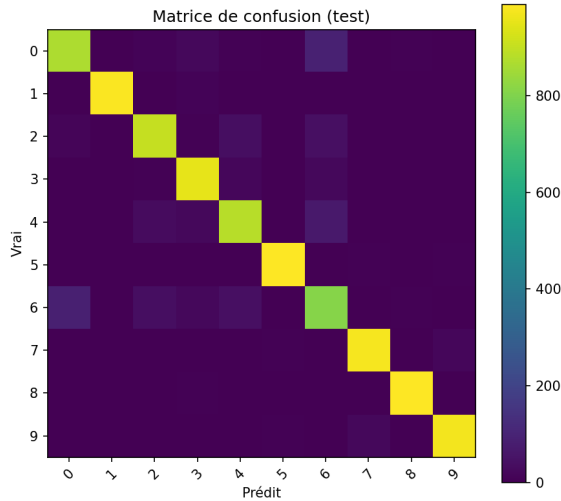
A wide variety of techniques has been applied to this dataset, ranging from traditional machine learning methods to advanced deep learning models. This paper provides a comparative analysis of these methods, focusing on the impact of different optimization algorithms. Our experiments show that RMSprop achieved the highest accuracy (93.17%) on our SimpleCNN architecture.

1 Introduction

Image classification is a fundamental challenge in computer vision. A popular and modern benchmark for this task is the Fashion-MNIST dataset, which provides a more complex challenge than the traditional MNIST. This

2 HOG with Support Vector Machine (SVM)

One common "traditional" approach involves using the Histogram of Oriented Gradients (HOG) feature descriptor with an SVM classifier. This method first requires pre-processing the images to reduce noise and improve recognition rates. The HOG descriptor then captures the shape and appearance of an image by dividing it into small cells and computing the directions of the contours (gradients) within them. The choice of cell size is a critical parameter. Studies show that a [4x4] cell size (creating 1296 features) is a good choice because it contains enough shape information and a limited dimensionality, which speeds up training. This is preferable to a [2x2] cell size (6084 features), which is too long, or an [8x8] cell size (144 features), which is not detailed enough and risks losing important contour information. When classified with an SVM—a model successful in many fields like face recognition—this HOG method achieved a test accuracy of (86.53%). A noted weakness of this approach was its difficulty in separating visually similar classes, such as 'Shirt' and 'T-shirt/top' in referenced paper 148 shirts were misclassified as tshirt.



3 k-Nearest Neighbors (k-NN) and Logistic Regression

Other studies have explored traditional machine learning models, often using dimensionality reduction to make them feasible. A study by Hu and Mulat focused on k-NN and Logistic Regression for the Fashion-MNIST dataset. Preprocessing with PCA The study noted that the high dimensionality of the data (784 features) resulted in long training times. To solve this, they applied Principal Component Analysis (PCA), a technique that projects the dataset into a lower-dimensional space.

- They first tested preserving (95%) of the variance, which reduced the data to 187 features.

- They also tested preserving (99%) of the variance, resulting in 458 features. They chose the 458-feature version, as it better-preserved image quality while still significantly reducing computational cost. Model Comparison The authors then trained two different types of models on this compressed 458-feature dataset:

1. Logistic Regression: This model attempts to find the best parameters (θ) to minimize a cost function $J(\theta)$. The study tested different optimizers to find this minimum, including SAG (Stochastic Average Gradient) and L-BFGS-B8. Their best logistic regression models achieved an accuracy of (85.25%).

2. k-Nearest Neighbors (k-NN): This algorithm classifies an image by taking a "vote" from its 'k' closest neighbors in the feature space. The study tested two different distance metrics:

- o Manhattan (L1) distance: The sum of the absolute differences along each feature axis.

- o Euclidean (L2) distance: The direct linear distance between two points. k-NN Results The k-NN classifier using Euclidean (L2) distance with a k-value of 6 produced the best result on their validation set¹³. The final accuracy on the test set for this k-NN model was (85.1%). This result is very similar to the best logistic regression model,

confirming that k-NN is a strong baseline, though it still falls slightly short of the (86.53%) accuracy achieved by the HOG+SVM method.

Their best results:

- Logistic Regression (SAG / L-BFGS-B): 85.25% accuracy.
- k-NN (L2, k=6): 85.1% accuracy.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)}))] \quad (1)$$

4 Advanced Deep Learning: ResNet and PyramidNet

For state-of-the-art performance, more complex architectures like Deep Residual Networks (ResNet) have been explored. A significant problem with very deep neural networks is that their performance can saturate or even degrade as more layers are added. The ResNet architecture was designed specifically to solve this issue. It introduces identity mapping (or "skip connections"), which allows the network to learn a residual function $F(x)$ ³. The final output of a block becomes $H(x) = F(x) + x$, which makes it possible to effectively train networks with hundreds of layers.

The study also investigated alternatives to simply increasing depth:

- Wider Networks: Instead of only going deeper, models like the Wide Residual Network (WRN) increase the "width" of the network (the number of filters/channels in the convolutional layers) .

- PyramidNet: This model was used as an efficient alternative. Instead of having a large number of filters at the beginning, it gradually widens the network by adding more filters in the deeper layers, resembling a pyramid.

Advanced Optimization Techniques This study also highlights that top performance comes from advanced training strategies, not just architecture. 1. WarmUp Strategy: Instead of starting with a high learning rate and decreasing it, this method uses a gradually increasing learning rate at the very beginning of training. This "warm-up" helps stabilize the model during the initial, sensitive phases.

2. Data Augmentation: To reduce overfitting, the training data was "augmented" by creating new, artificial images. This included methods like Resize, RandomCrop, RandomHorizontalFlip.

By combining a PyramidNet with these advanced data augmentation and optimization strategies , the study achieved a final accuracy of (96.21%), the highest reported in our references

5 Artificial Neural Network (ANN)

The study by Sumera et al. also provides a baseline for a traditional Artificial Neural Network (ANN). This model serves as a direct comparison to the more modern CNN architecture.

The ANN architecture used was a sequential model with three dense layers. The process begins by flattening the $28 \times 28 \times 1$ input images into a one-dimensional vector of 784 features.

- Hidden Layer 1 (FC1): A dense layer with 3,000 neurons.
- Hidden Layer 2 (FC2): A dense layer with 1,000 neurons.
- Output Layer: A final dense layer with 10 neurons (one for each class).

The primary drawback of this model, as highlighted in the paper, is the enormous number of trainable parameters required. The calculation for the parameters is as follows:

1. FC1 (Input $\rightarrow$ H1): $(784 \text{ inputs} \times 3000 \text{ neurons}) + 3000 \text{ biases} = 2,355,000 \text{ parameters}$
2. FC2 (H1 $\rightarrow$ H2): $(3000 \text{ neurons} \times 1000 \text{ neurons}) + 1000 \text{ biases} = 3,001,000 \text{ parameters}$
3. Output (H2 $\rightarrow$ Out): $(1000 \text{ neurons} \times 10 \text{ outputs}) + 10 \text{ biases} = 10,010 \text{ parameters}$

6 Convolutional Neural Network (CNN)

The Convolutional Neural Network (CNN) is a specialized architecture for image tasks. Unlike a traditional ANN, a CNN has significant advantages:

It drastically reduces the number of trainable parameters, which speeds up the training process.

It is designed to automatically learn relevant features at different levels from the image by using filters, similar to how human vision works.

The architecture used in this study is a sequential model. The core component is the convolution layer, which performs feature extraction. This is followed by a pooling layer (like max pooling) to downsample the feature map and extract the most prominent values. Finally, fully connected layers are used at the end to perform the classification.

Each convolution layer is followed by a ReLU (Rectified Linear Unit) activation function. This step is critical as it breaks the linearity and allows the network to learn complex patterns.

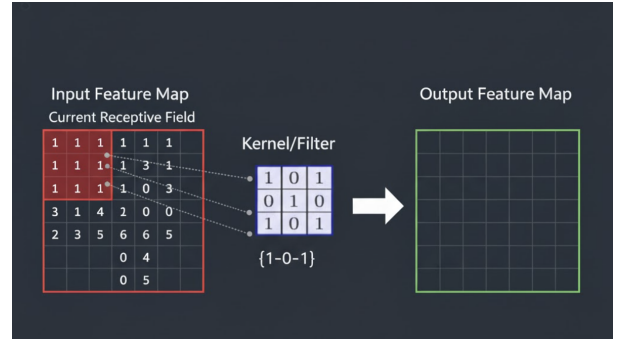


Figure 1: filter conv

3.2. Parameter Calculation The efficiency of the CNN is clear when calculating its parameters, as shown in the source study. The formula for parameters in a convolution layer is: $(\text{filterwidth} * \text{filterheight} * \text{inputchannels} + 1) * \text{numfilters}$. The +1 is for the bias.

Conv 1: $((3*3*1) + 1) * 32 = 320 \text{ parameters}$.

Conv 2: $((3*3*32) + 1) * 64 = 18,496 \text{ parameters}$. (Note: It's 32 input channels from Conv 1).

FC 1 (Dense): The model flattens the feature map to 1600 neurons. $(1600 * 64) + 64 = 102,464 \text{ parameters}$.

FC 2 (Output): $(64 * 10) + 10 = 650 \text{ parameters}$.

Total Trainable Parameters: $320 + 18,496 + 102,464 + 650 = 121,930$

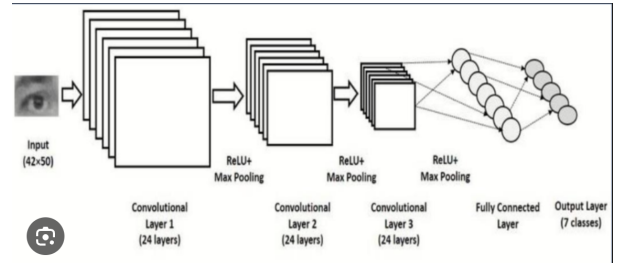


Figure 2: CNN

The ANN results in a total of 5,366,010 trainable parameters is over 44 times more than the 121,930 parameters required by the CNN model in the same study.

While the ANN's overall accuracy was lower than the CNN's, the study found a curious exception: when using the SGD and Adagrad optimizers, the ANN achieved a higher testing accuracy than the CNN 13.

However, when paired with the more advanced RM-Sprop and Adam optimizers, the CNN was significantly more accurate, establishing it as the superior model overall.

7 Optimizer Comparison and Results

To find the most effective training method for our SimpleCNN architecture, we conducted experiments using four different optimization algorithms. The goal was to

compare their final accuracy on the Fashion-MNIST test set after 12 epochs.

4.1 Optimizer Performance • Adagrad: This optimizer performed the worst in our study, achieving an accuracy of only (88%). As noted in the literature , Adagrad’s main drawback is that it continually accumulates the sum of squared gradients in its denominator. This causes the learning rate to shrink and eventually become too small, effectively stopping the model from learning new information.

metrics_baseline

epoch	train_loss	test_accuracy	test_f1_macro
1	0.760857	0.792000	0.781993
2	0.558827	0.814900	0.809456
3	0.508743	0.827300	0.821201
4	0.477902	0.837900	0.833196
5	0.457807	0.844600	0.841170
6	0.439467	0.850000	0.847953
7	0.428081	0.853700	0.851985
8	0.418141	0.858000	0.856539
9	0.406267	0.858700	0.857637
10	0.400544	0.860500	0.858686
11	0.393947	0.862500	0.861268
12	0.388738	0.865700	0.864862

Figure 3: adagrad acc

• Stochastic Gradient Descent (SGD): Our baseline CNN model paired with SGD (using a learning rate of 0.01 and momentum) achieved a strong accuracy of (91.68%). Standard SGD converges "noisily" as it updates weights one sample at a time. Adding momentum helps to smooth this convergence by giving higher weight to the recent, consistent gradient directions.

metrics_baseline

epoch	train_loss	test_accuracy	test_f1_macro
1	0.876046	0.830200	0.825916
2	0.435409	0.851200	0.849058
3	0.368413	0.876900	0.876734
4	0.331208	0.885900	0.884710
5	0.303832	0.890500	0.891075
6	0.281495	0.899200	0.898754
7	0.265197	0.899900	0.900566
8	0.249410	0.907200	0.906686
9	0.235997	0.908300	0.907893
10	0.224764	0.911200	0.911175
11	0.214626	0.911800	0.911382
12	0.206625	0.916800	0.916709

Figure 4: SGD acc

• Adam (Adaptive Moment Estimation): Adam is a sophisticated optimizer that combines the adaptive learning rate of RMSprop with the smoothing of momentum . It adapts the learning rate for each parameter, giving more importance to recent gradients. While Adam is a common high-performing default, in our experiment, it yielded a slightly lower accuracy than our final model.

metrics_baseline

epoch	train_loss	test_accuracy	test_f1_macro
1	0.527833	0.878000	0.875384
2	0.320369	0.892400	0.890514
3	0.269559	0.907600	0.907237
4	0.239767	0.913500	0.913207
5	0.216498	0.913800	0.914572
6	0.199152	0.922400	0.922495
7	0.186147	0.922900	0.922561
8	0.171275	0.924400	0.923977
9	0.160257	0.924900	0.924791
10	0.151018	0.928000	0.927743
11	0.137897	0.926400	0.926201
12	0.128891	0.929100	0.929165

Figure 5: Adam Acc

- RMSprop (Root Mean Square propagation): This optimizer was the top performer in our study, achieving a final test accuracy of (93.17%). RMSprop directly addresses the flaw in Adagrad. Instead of letting the denominator accumulate forever, it uses an exponentially weighted moving average of the squared gradients. This allows it to remain adaptive without its learning rate vanishing.

metrics_baseline

epoch	train_loss	test_accuracy	test_f1_macro
1	0.537486	0.869500	0.868147
2	0.308512	0.902300	0.901485
3	0.257077	0.909300	0.909315
4	0.226217	0.916200	0.914054
5	0.205307	0.924900	0.924363
6	0.188052	0.914700	0.913569
7	0.177844	0.922100	0.922688
8	0.167232	0.920300	0.921475
9	0.160336	0.928400	0.927730
10	0.153321	0.929500	0.928693
11	0.145650	0.931000	0.930587
12	0.142555	0.931700	0.931814

Figure 6: RMSprop acc

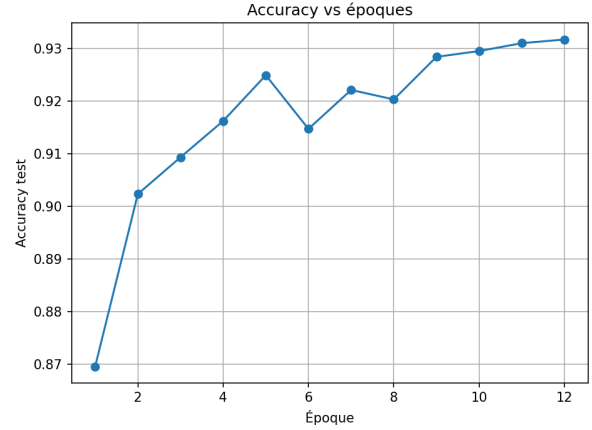


Figure 7: curve acc

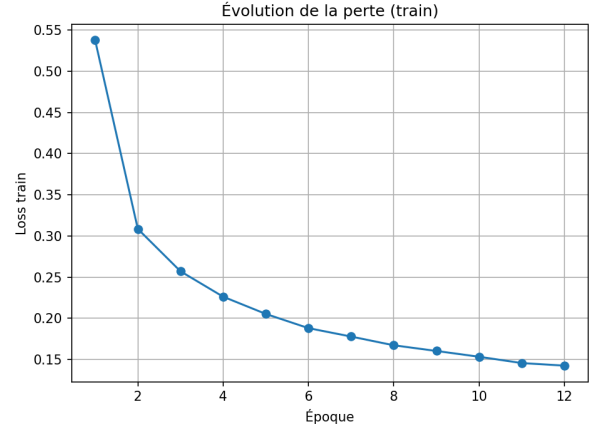


Figure 8: loss

7.1 Analysis

The most interesting finding from our experiment is that RMSprop outperformed Adam. While Adam is often considered the state-of-the-art default, our results show that the combination of its specific hyperparameters (using $lr=0.001$ and $\alpha=0.9$, as seen in our code) provided a more stable and effective convergence for our specific SimpleCNN architecture than the Adam optimizer did. This suggests that the added momentum component of Adam was not as beneficial for this problem, and the simpler, adaptive learning rate of RMSprop was sufficient and more effective.

8 Conclusion

This paper conducted a comprehensive comparative analysis to identify the most effective classification method for the Fashion-MNIST dataset. Our review of existing literature covered a range of models, from traditional machine learning approaches to advanced deep learning architectures. Convolutional Neural Networks (CNNs) are far more efficient, requiring only 122,000 parameters in a comparable study. At the high end, complex architectures like PyramidNet, combined with advanced data augmentation, can achieve state-of-the-art accuracy of 96.21%. Our own experimental work focused on optimizing a baseline SimpleCNN model. We confirmed the findings in the literature regarding optimizers: • Adagrad performed the worst (88%) accuracy in our test, which aligns with its known flaw of having a learning rate that vanishes over time. • SGD (with momentum) and Adam provided strong, high-performing results. • The best performance in our experiment was achieved by RMSprop, which yielded a final test accuracy of 93.17%. This key finding demonstrates that while Adam is an excellent general-

purpose optimizer, it is not universally superior. For our specific SimpleCNN architecture, the adaptive learning rate of a well-tuned RMSprop provided a more effective convergence. Ultimately, this study confirms that Convolutional Neural Networks are the superior architecture for this task, and their performance is critically dependent on a properly chosen and configured optimizer.

9 Broader Impacts and Limitations

While image classification has achieved high levels of accuracy, it should always include human intervention. This is because no model can achieve (100%) accuracy; a model that claims to do so is likely overfitted to its training data and will fail on unpredictable, real-world inputs. A recent example illustrates this risk: a school security system reportedly misidentified a pack of chips as a weapon, triggering a police response. It was later revealed to be a mistake by the system. This incident highlights that while machine learning is a powerful and beneficial tool, it lacks true human context and understanding. Therefore, any real-world action based on a model's prediction, especially one concerning safety, must be verified by human intervention.

10 References

- 1-Sumera, Sirisha R, Anjum N, Vaidehi K (2022) Implementation of CNN and ANN for Fashion-MNIST-Dataset using Different Optimizers.
- 2-Tang Y, Cui H and Liu S (2020) Optimal Design of Deep Residual Network Based on Image Classification of Fashion-MNIST Dataset.
- 3-Hossain MA, Sajib MSA (2019) Classification of Image using Convolutional Neural Network (CNN).
- 4-Greeshma K V, Sreekumar K (2019) Fashion-MNIST Classification Based on HOG Feature Descriptor Using SVM.
- 5-Hu L, Mulat N (2017) Classify Fashion Products from Fashion-MNIST Thumbnail Images.